说明文档

B/S架构

B/S 指浏览器，服务器

特点：

1. 不需要安装，直接在浏览器上访问指定的网址就可以
2. 软件更新时，客户端不用更新
3. 软件可以跨平台，只要有浏览器们就可以使用
4. B/S架构的软件，客户端与服务器之间通信采用的是`http协议`，相对来说不安全。后来就有了`https`加密协议，改变了安全性，但是成本较昂贵。

> **关于 `Http` 的简单知识：**
>
> `Http（全称：Hyper Text Transfer Protocol）`是一个简单的请求-响应协议，它通常运行在TCP（传输控制协议（Transmission Control Protocol））之上。它指定了客户端可能发送给服务器什么样的消息以及得到什么样的响应。请求和响应消息的头以ASCII码形式给出；而消息内容则具有一个类似MIME的格式。
> HTTP协议的服务器端实现程序有httpd、nginx等，其客户端的实现程序主要是Web浏览器，例如Firefox、IE、Google chrome、Safari、Opera等，此外，客户端的命令行工具还有elink、curl等。Web服务是基于TCP的，因此为了能够随时响应客户端的请求，Web服务器需要监听在80/TCP端口。这客户端浏览器和Web服务器之间就可以通过HTTP协议进行通信了。浏览器会自动添加80端口号。

B/S架构下的前端和后端

那么前端是什么，后端是什么，他们之间又是怎么来完成交互的：

- 前端：

通过在页面上绑定的事件，获取到用户的信息，`把信息传递给后端`，让后端存储起来，同时把后端给的数据展示在页面上。

- 后端：

  当前端需要数据的时候，后端从数据库拿到数据给前端，同时把前端传进来的数据存到数据库里。注意的是：数据库是专门让后端进行数据的 增 删 改 查 的。并且`前端是不能够操作数据库`的，我们只有通过和后端的相互交互来实现共同的效果。

前端和后端的交互基于http协议（或https协议），前端在浏览器中使用POST，DELETE，PUT和GET四种请求方式分别对指定的URL资源进行增删改查操作。

本Demo是一个基于Http的前后端分离的B/S架构`，B/S架构，即浏览器（Browser）承担与用户交互的工作（包括构建DOM树，解析javascript，渲染CSS等工作），服务器（Server`）存储用户数据和操作用户数据的方法。

前端服务和后端服务

一般来说，服务（`server`）是指web服务（`web server`），前端无非就是把 `html`、`css`、`js` 一股脑丢给浏览器让它渲染成一个供用户操作的页面，那为什么我们要启动一个前端服务呢？

在以前`JSP`盛行的年代，的确很少有前台服务的说法，那时候渲染工作是由后台完成的，后端工程师将要展现给用户的数据插入html文件中，然后传递静态页面给前台，前台再交给浏览器进行解析，当用户操作页面元素时，再由javascript捕获事件对象、事件类型，根据类型执行相应函数，进而实现页面的动态变化。

而现在随着Angular、React、Vue等响应式JavaScript框架的出现，带动了前后端分离开发的热潮，几乎所有和用户交互相关的操作如今都在前端完成，而后端只需要专注于性能优化，换句话说，前后的边界逐渐向前端偏移。

传统的前端视图展示只依赖 html、css、js 三驾马车（关于HTML/CSS/JS是如何在浏览器中渲染成我们看到的页面，可以参考https://mp.weixin.qq.com/s/0jXQFRzFDfuphjU40x8LfA），但在前后端分离的模式下，服务被分为了`前端服务`和`后端服务`两部分，前端服务为用户提供无模型的视图（无数据）；后端服务为前端服务提供接口。浏览器向前端服务请求视图，通过视图中包含的JavaScript函数发起接口请求获取模型。

前后端分离的项目开发，有利于团队并行开发，提高生产效率。在前后端分离的开发模式中，我们将HTTP请求转移到前端服务器上，降低服务器的负荷，使视图获取后端模型失败也能呈现。但前后端分离的开发模式却不适用于所有的项目，当项目比较小时无需分离，这将使项目变得更加复杂。

启动后端服务前需要知道的maven

在了解Maven之前，我们先来看看一个Java项目需要的东西。

1. 首先，我们需要确定引入哪些依赖包。例如，如果我们需要用到commons logging，我们就必须把commons logging的jar包放入classpath。如果我们还需要log4j，就需要把log4j相关的jar包都放到classpath中。这些就是依赖包的管理。
2. 其次，我们要确定项目的目录结构。例如，`src`目录存放Java源码，`resources`目录存放配置文件，`bin`目录存放编译生成的`.class`文件。
3. 此外，我们还需要配置环境，例如JDK的版本，编译打包的流程，当前代码的版本号。
4. 最后，除了使用Eclipse这样的IDE进行编译外，我们还必须能通过命令行工具进行编译，才能够让项目在一个独立的服务器上编译、测试、部署。

这些工作难度不大，但是非常琐碎且耗时。如果每一个项目都自己搞一套配置，肯定会一团糟。我们需要的是一个标准化的Java项目管理和构建工具。

Maven就是是专门为Java项目打造的管理和构建工具，它的主要功能有：

- 提供了一套标准化的项目结构；
- 提供了一套标准化的构建流程（编译，测试，打包，发布……）；
- 提供了一套依赖管理机制。

Maven项目结构

一个使用Maven管理的普通的Java项目，它的目录结构默认如下：

```
a-maven-project
├── pom.xml
├── src
|   ├── main
|   |   ├── java
|   |   └── resources
|   └── test
|       ├── java
|       └── resources
└── target
```

项目的根目录`a-maven-project`是项目名，它有一个项目描述文件`pom.xml`，存放Java源码的目录是`src/main/java`，存放资源文件的目录是`src/main/resources`，存放测试源码的目录是`src/test/java`，存放测试资源的目录是`src/test/resources`，最后，所有编译、打包生成的文件都放在`target`目录里。这些就是一个Maven项目的标准目录结构。

所有的目录结构都是约定好的标准结构，我们千万不要随意修改目录结构。使用标准结构不需要做任何配置，Maven就可以正常使用。

我们再来看最关键的一个项目描述文件`pom.xml`，它的内容长得像下面：

```
<project ...>
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.itranswarp.learnjava</groupId>
    <artifactId>hello</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>
    <properties>
        ...
```

```
    </properties>
    <dependencies>
        <dependency>
            <groupId>commons-logging</groupId>
            <artifactId>commons-logging</artifactId>
            <version>1.2</version>
        </dependency>
    </dependencies>
</project>
```

其中，`groupId`类似于Java的包名，通常是公司或组织名称，`artifactId`类似于Java的类名，通常是项目名称，再加上`version`，一个Maven工程就是由`groupId`，`artifactId`和`version`作为唯一标识。我们在引用其他第三方库的时候，也是通过这3个变量确定。例如，依赖`commons-logging`：

```
<dependency>
    <groupId>commons-logging</groupId>
    <artifactId>commons-logging</artifactId>
    <version>1.2</version>
</dependency>
```

使用`<dependency>`声明一个依赖后，Maven就会自动下载这个依赖包并把它放到classpath中。

如何启动后端服务

首先需要安装Maven

要安装Maven，可以从[Maven官网]下载最新的Maven 3.6.x，然后在本地解压，设置几个环境变量：

```
M2_HOME=/path/to/maven-3.6.x
PATH=$PATH:$M2_HOME/bin
```

Windows可以把`%M2_HOME%\bin`添加到系统Path变量中。

然后，打开命令行窗口，输入`mvn -version`，应该看到Maven的版本信息：

如果提示命令未找到，说明系统PATH路径有误，需要修复后再运行。

安装好maven后，我们进入项目目录，并输入启动命令

```
# 进入项目目录
PS D:\gitproject\restfulDemo\be-spring>
# 启动服务
PS D:\gitproject\restfulDemo\be-spring> mvn spring-boot:run
```

这样就启动成功了

```
r with driverClassName=com.mysql.jdbc.Driver was not found, trying direct instantiation.
2022-10-19 09:08:36.652  INFO 23660 --- [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - S
tart completed.
2022-10-19 09:08:36.678  INFO 23660 --- [           main] org.hibernate.dialect.Dialect            : HHH000400: Using
 dialect: org.hibernate.dialect.MySQL8Dialect
2022-10-19 09:08:37.059  INFO 23660 --- [           main] o.h.e.t.j.p.i.JtaPlatformInitiator       : HHH000490: Using
 JtaPlatform implementation: [org.hibernate.engine.transaction.jta.platform.internal.NoJtaPlatform]
2022-10-19 09:08:37.066  INFO 23660 --- [           main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA
EntityManagerFactory for persistence unit 'default'
2022-10-19 09:08:37.227  WARN 23660 --- [           main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-
in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configu
re spring.jpa.open-in-view to disable this warning
2022-10-19 09:08:37.424  INFO 23660 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started o
n port(s): 8080 (http) with context path ''
2022-10-19 09:08:37.430  INFO 23660 --- [           main] c.e.a.AccessingDataMysqlApplication      : Started Accessin
gDataMysqlApplication in 2.558 seconds (JVM running for 2.858)
```

启动前端服务前需要知道的node、npm和yarn

node

node（Node.js） 是一个开源和跨平台的 JavaScript 运行时环境。 要运行一个JavaScript程序，只需要将代码片段另存为 test.js，并在终端中输入`node test.js` 就能运行该程序。

Node.js 应用程序在单个进程中运行，无需为每个请求创建新的线程。Node.js 在其标准库中提供了一组异步的 I/O 原语，以防止 JavaScript 代码阻塞，通常，Node.js 中的库是使用非阻塞范式编写的，使得阻塞行为成为异常而不是常态。

当 Node.js 执行 I/O 操作时（比如从网络读取、访问数据库或文件系统），Node.js 将在响应返回时恢复操作（而不是阻塞线程和浪费 CPU 周期等待）。

这允许 Node.js 使用单个服务器处理数千个并发连接，而不会引入管理线程并发（这可能是错误的重要来源）的负担。

Node.js 具有独特的优势，因为数百万为浏览器编写 JavaScript 的前端开发者现在无需学习完全不同的语言，就可以编写除客户端代码之外的服务器端代码。

npm

npm 全称是 Node Package Manager，它存储了大量js库，npm 帮助 Node.js 生态系统蓬勃发展，现在 npm 仓库托管了超过 1,000,000 个开源包，全部都可以自由使用。

为什么我们需要一个包管理工具呢？因为我们在Node.js上开发时，会用到很多别人写的JavaScript代码。如果我们要使用别人写的某个包，每次都根据名称搜索一下官方网站，下载代码，解压，再使用，非常繁琐。于是一个集中管理的工具应运而生：大家都把自己开发的模块打包后放到npm官网上，如果要使用，直接通过npm安装就可以直接用，不用管代码存在哪，应该从哪下载。

更重要的是，如果我们要使用模块A，而模块A又依赖于模块B，模块B又依赖于模块X和模块Y，npm可以根据依赖关系，把所有依赖的包都下载下来并管理起来。否则，靠我们自己手动管理，既麻烦又容易出错。

yarn

Yarn是由Facebook、Google、Exponent 和 Tilde 联合推出了一个新的 JS 包管理工具，正如官方文档中写的，Yarn 是为了弥补 npm 的一些缺陷而出现的。

npm的缺陷：

1. `npm install`的时候**巨慢**。特别是新的项目拉下来要等半天，删除node_modules，重新install的时候依旧如此。
2. 同一个项目，安装的时候**无法保持一致性**。由于package.json文件中版本号的特点，下面三个版本号在安装的时候代表不同的含义。

yarn的优点：

1. 并行安装：无论 npm 还是 Yarn 在执行包的安装时，都会执行一系列任务。npm 是按照队列执行每个 package，也就是说必须要等到当前 package 安装完成之后，才能继续后面的安装。而 Yarn 是同步执行所有任务，提高了性能。
2. 离线模式：如果之前已经安装过一个软件包，用Yarn再次安装时之间从缓存中获取，就不用像npm那样再从网络下载了。

> 使用 npm 和 yarn 时要注意，严禁 npm 和 yarn 混用，极易造成package丢失问题

如何启动前端服务

- 安装node

参考官方网站http://nodejs.cn/download/

npm已经在Node.js安装的时候顺带装好了。在终端输入`npm -v`，应该看到类似的输出：

```
(base) PS D:\restfulDemo> npm -v
6.14.4
(base) PS D:\restfulDemo>
```

- 安装yarn

参考官方网站https://yarn.bootcss.com/docs/install/#windows-stable

配置好环境变量后，在终端输入`yarn -v`，应该看到类似的输出：

```
(base) PS D:\restfulDemo> yarn -v
1.22.19
```

安装好yarn后，我们进入项目目录，并输入启动命令`yarn start`

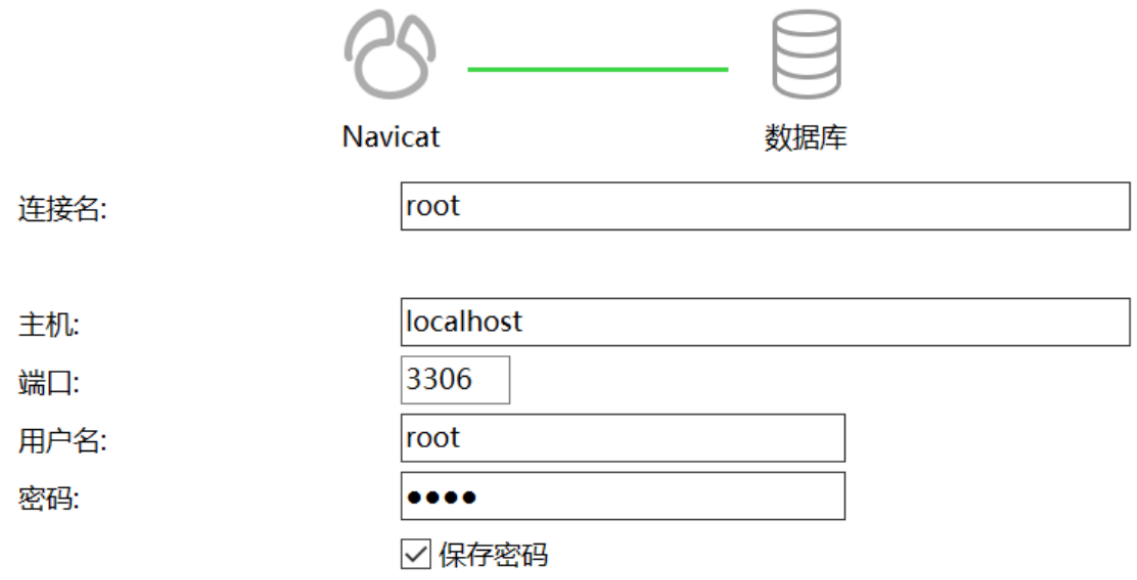
```
# 进入项目目录
PS D:\gitproject\restfulDemo\fe\my-app>
# 安装依赖
PS D:\gitproject\restfulDemo\fe\my-app> yarn install
# 启动服务
PS D:\gitproject\restfulDemo\fe\my-app> yarn start
```

这样就启动成功了

```
Compiled successfully!

You can now view my-app in the browser.

  Local:            http://localhost:3000
  On Your Network:  http://192.168.1.4:3000

Note that the development build is not optimized.
To create a production build, use yarn build.
```

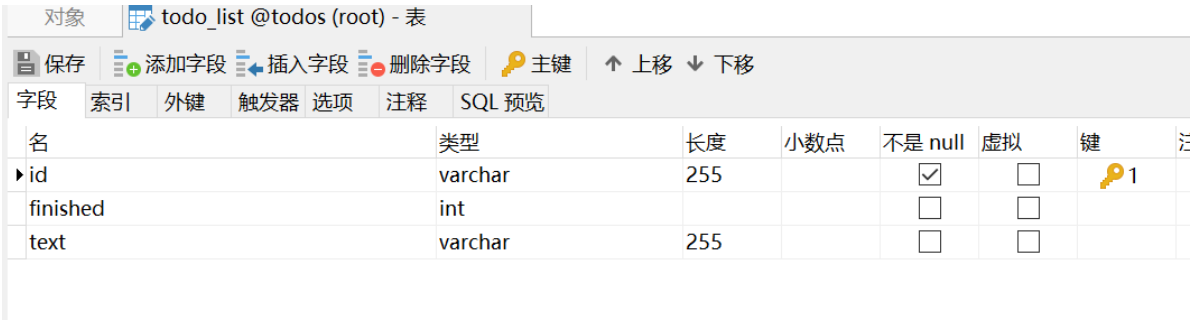
数据库配置

账号 / 密码 root

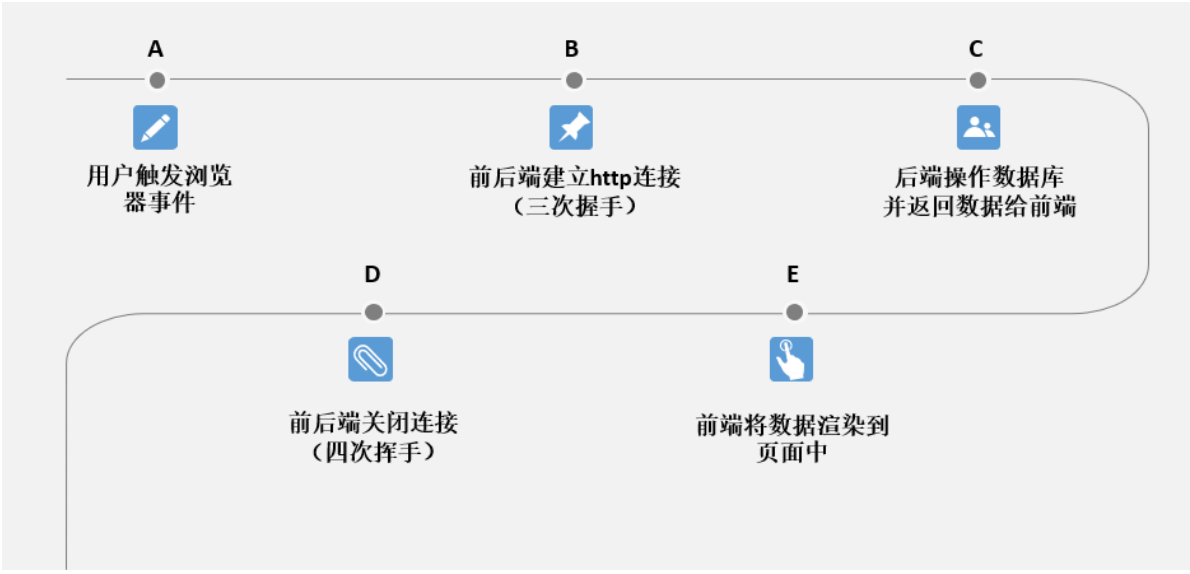


数据库 todos（字符集utf8排序规则general_ci）

表 todo_list



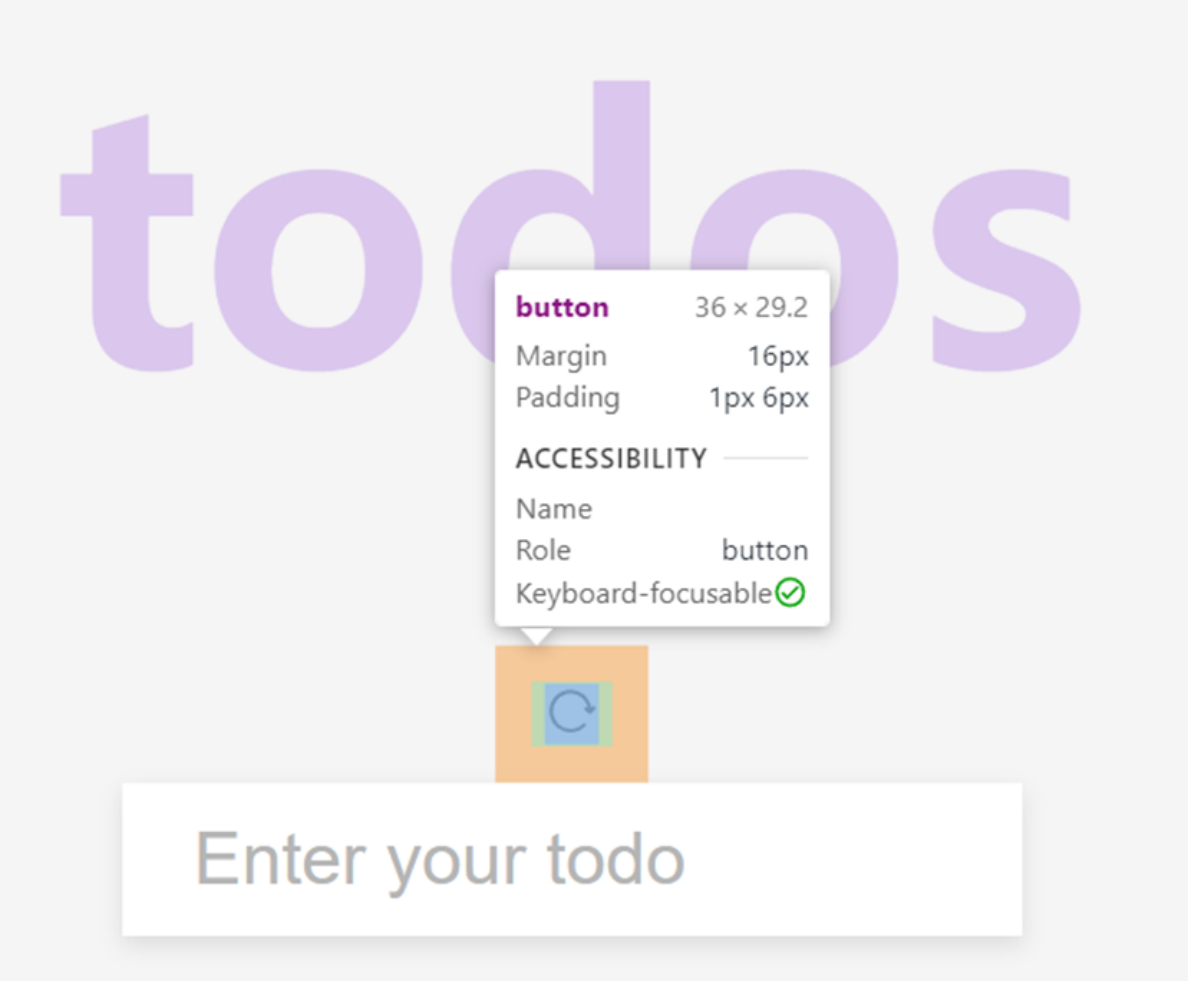
前端、后端、数据库交互



以获取数据库中所有todos为例，说明前端、后端、数据库交互的过程：

1. 进入页面，获取button元素
2. 点击button，执行绑定在button上的函数
3. 函数中发起http请求（包含请求路径、请求方法、参数类型、请求体等）
4. 前后端建立连接（三次握手）
5. 后端根据请求路径路由到对应的controller函数，并执行该函数
6. 如果函数中包含了对数据库的操作，则后端将与数据库建立连接
7. 后端从数据库中拿到对应数据（或修改数据库中的数据）
8. 后端拿到数据后设置状态码与响应体，并返回响应
9. 前端收到响应
10. 前后端关闭连接（四次挥手）
11. 前端将数据插入html，并触发浏览器的重新渲染

进入页面，获取button元素



这个button在前端代码中：

```
<button onClick={getTodos}><IconFont type={"icon-shuaxin"}/></button>
```

这个代码中绑定了`onClick`函数，表示当该按钮被点击时，调用`getTodos`方法

点击button，执行绑定在button上的函数

这是`getTodos`方法：

```
const getTodos = () => {

 axios.get(`http://127.0.0.1:7001/api/getTodos`).then((res) => {
      const data = res.data
      setLocalTodos(data)
    })
}
```

可以从这个函数中看到，函数通过axios方法向`http://127.0.0.1:7001/api/getTodos`发起GET请求

函数中发起http请求（包含请求路径、请求方法、参数类型、请求体等）

请求路径：

`http://127.0.0.1:7001/api/getTodos`

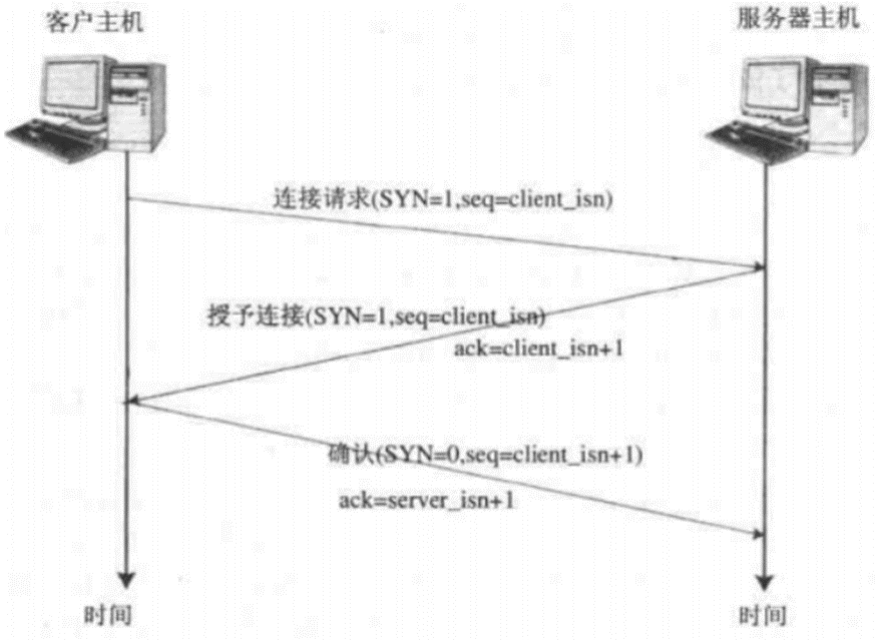
请求方法：

`GET`

参数类型：

`这次请求没有参数`

前后端建立连接（三次握手）



客户主机就是localhost（http://127.0.0.1:3000）

服务主机就是也是localhost（http://127.0.0.1:8080）

> 注：虽然客户机和服务器都是localhost，但它们并不在同一个端口上，所以属于不同域

后端根据请求路径路由到对应的controller函数，并执行该函数

请求路径`/api/getTodos`对应的函数为`controller.demo.getTodos`

```
module.exports = app => {
  const { router, controller } = app;
  router.get('/api/getTodos', controller.demo.getTodos);
  router.post('/api/addTodo', controller.demo.addTodo);
  router.post('/api/finishTodo', controller.demo.finishTodo);
  router.post('/api/delTodo', controller.demo.delTodo);
};
```

如果函数中包含了对数据库的操作，则后端将与数据库建立连接

后端路由中地址为`/api/getTodos`对应的函数为`getTodos`

```
class DemoController extends Controller {
  async getTodos() {
    // 请求数据库
    const res = await this.ctx.model.Todo.findAll()
    // 返回结果
    this.ctx.status = 200
    this.ctx.body = res
  }
}
```

可以看到，`getTodos`中包含了请求数据库，数据库中的操作为`findAll()`

后端从数据库中拿到对应数据（或修改数据库中的数据）

model层描述了数据库中的数据类型

当后端要与数据库建立连接时，会调用该层

```
module.exports = app => {
  const { STRING, INTEGER } = app.Sequelize;

  const Todo = app.model.define('todo_list', {
    id: { type: STRING, primaryKey: true },
    text: STRING(50),
    finished: INTEGER(1),
  },{
    timestamps: false, //自动增加创建时间
    tableName: 'todo_list' //设置表名称
  });

  return Todo;
};
```

从后端中获取的数据存在了`res`变量中

```
// 请求数据库
const res = await this.ctx.model.Todo.findAll()
```

后端拿到数据后设置状态码与响应体，并返回响应

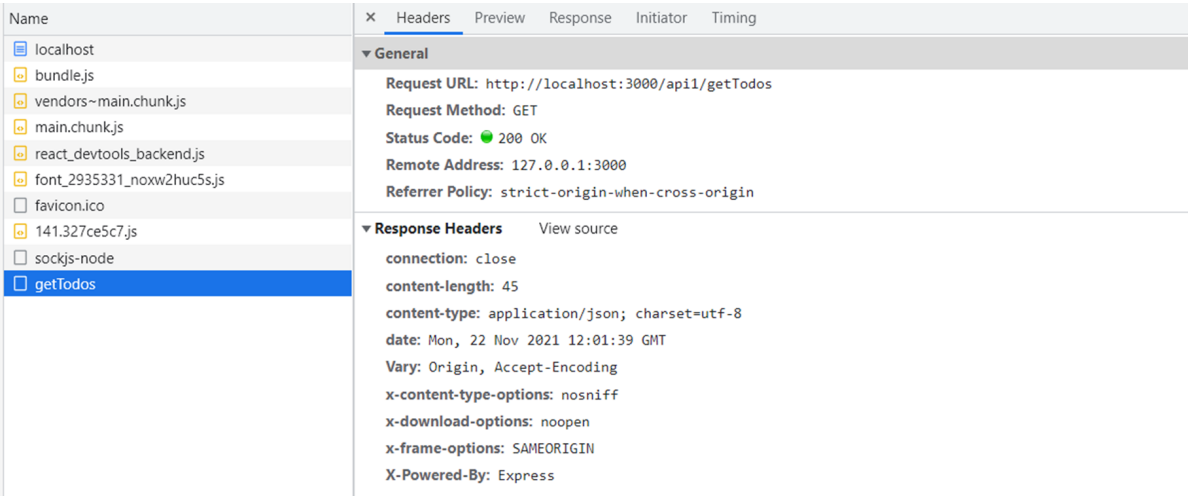
设置状态码200，表示请求成功处理

并将`res`作为请求体返回给前端

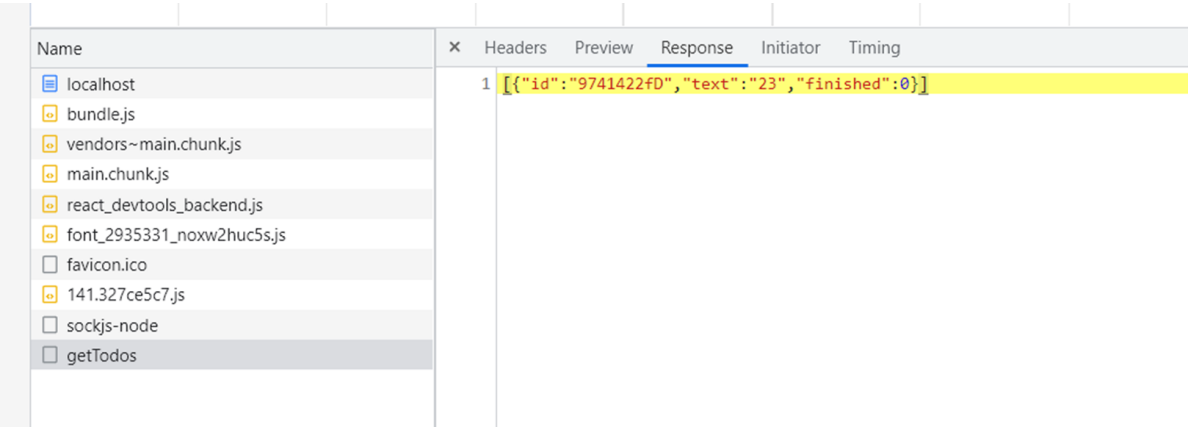
```
// 返回结果
this.ctx.status = 200
this.ctx.body = res
```

前端收到响应

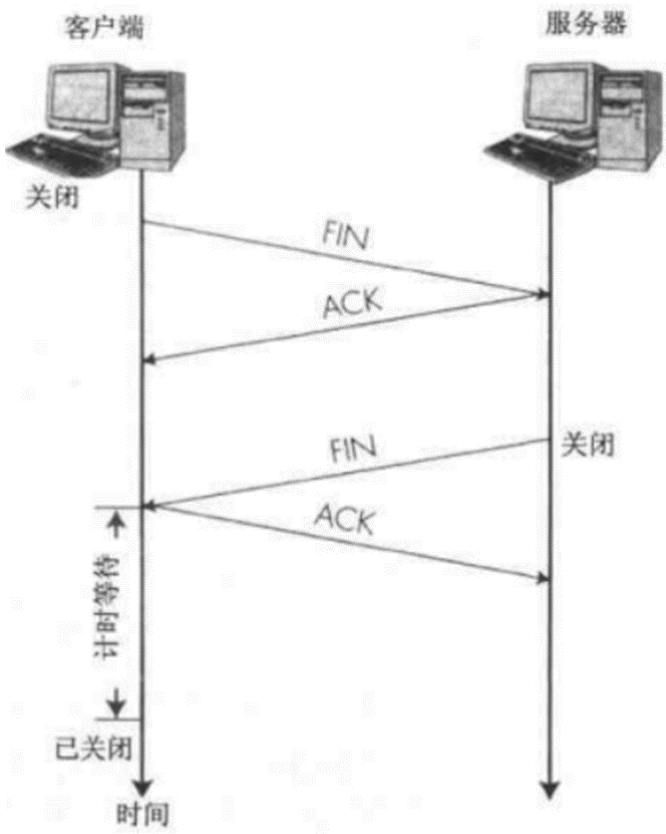
通过浏览器控制台，我们可以看到后台响应头部



以及后台返回的响应体：



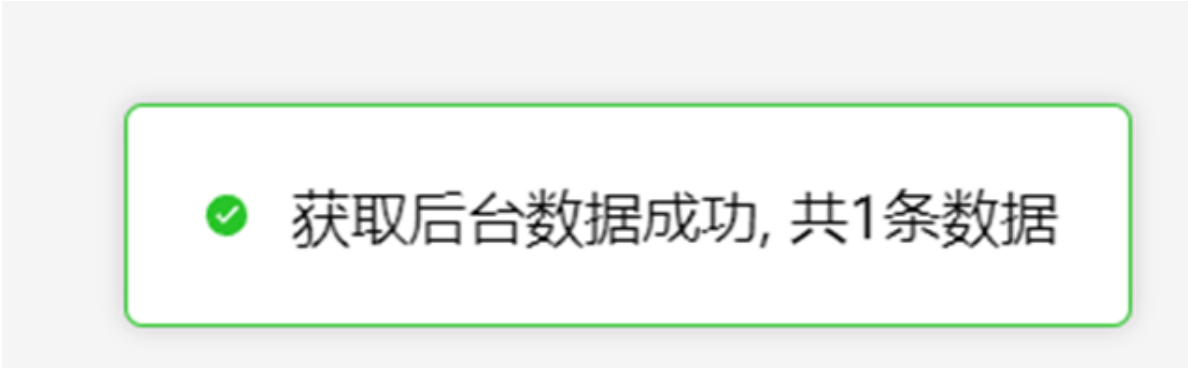
前后端关闭连接（四次挥手）

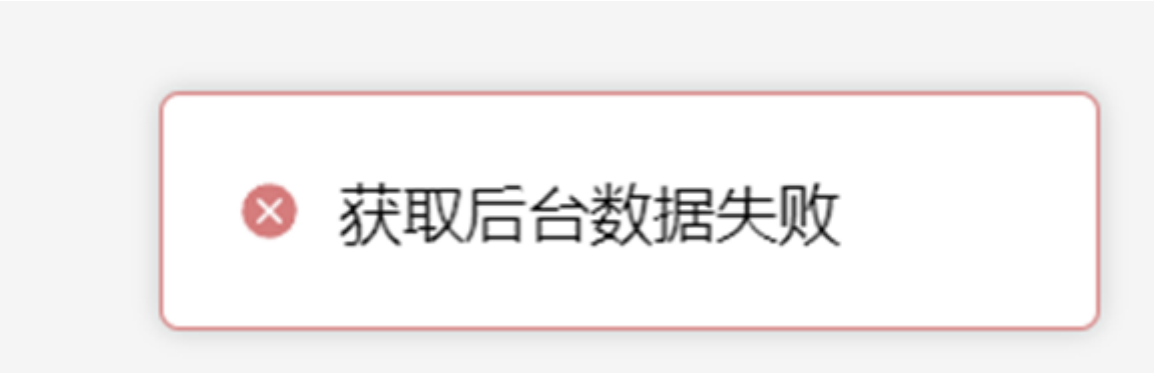


前端将数据插入html，并触发浏览器的重新渲染

如果数据获取成功，将执行以下代码

```
setLocalTodos(data)
setNotification([
    ...notification,
    {
        id: generateId(),
        type: 'success',
        message: `获取后台数据成功，共${data.length}条数据`
    }
])
```



如果数据获取失败，将执行以下代码

```
setNotification([
    ...notification,
    {
        id: generateId(),
        type: 'error',
        message: '获取后台数据失败'
    }
])
```